

FOCP II Assignments

Submission on : GitHub repository “**FOCP-II-Assignments**”:

Dear Students,

- Solve all assigned problems on HackerEarth within the deadline
- Submit the following on the GitHub repository “**FOCP-II-Assignments**”:
 1. Screenshot of successful submission (must show username, problem name, status, date & time)
 2. Your code file (.cpp)
- Follow file naming format strictly:
 - i. **QuestionName.cpp**
 - ii. **QuestionName.png**
- Ensure your code is original; copying will result in **0 marks**
- Late submissions will not be accepted
- Incomplete or unclear submissions will not be evaluated
- Evaluation will be based on correctness, code quality, and proper submission

Assignment 1 - Deadline 12-04-2026

1. https://www.hackerearth.com/problem/algorithm/turn-off-lights-be46dc6c/?source=list_view
2. <https://www.hackerearth.com/problem/algorithm/count-pairs-01cd4368/>

Assignment 2 - Deadline 19-04-2026

3. https://www.hackerearth.com/problem/algorithm/help-alice-to-find-the-parity-f65a1d30/?source=list_view
4. https://www.hackerearth.com/problem/algorithm/too-chocolatey-232e97e0/?source=list_view

Assignment 3 - Deadline 26-04-2026

5. Problem name: Movie Tickets

Problem statement

Assume that there are 100 slots available for every movie.

Given Q queries of the following types:

- **BOOK $X Y$**
 - Allow a customer with a unique ID X to book a ticket for a movie with the movie ID Y .
 - If the customer has already booked a ticket for the same movie or the movie is sold out, return *false*.
 - Otherwise, mark the ticket as booked and return *true*.
- **CANCEL $X Y$**
 - Allow a customer with a unique ID X to cancel a booked ticket for a movie with the movie ID Y .
 - If the customer has not booked a ticket for the movie, return *false*.
 - Otherwise, mark the ticket as canceled and return *true*.
- **IS_BOOKED $X Y$**
 - Check if a customer with a unique ID X has booked a ticket for the movie with the movie ID Y .
 - Return *true* if booked; otherwise, return *false*.
- **AVAILABLE_TICKETS Y**
 - Return the number of available tickets for the movie with the movie ID Y .

Class description

- **MovieTicket:** Represents information about the users and movies
- **Public functions:** Refers to the types of queries (described above) that must be implemented

Input format for custom testing

Note: Use this input format if you are testing against custom input or writing code in a language where we don't provide boilerplate code.

- The first line contains a single integer Q , denoting the number of elements in the array.
- Each of the next Q lines contains the query (case-sensitive for strings).

Output format

Return the result for each function method based on the query.

6. Problem Name: Bank Balance

Problem statement

Given Q queries of the following types:

- **CREATE $X\ Y$**
 - Create a user account with a user ID X and an initial balance of Y dollars.
 - If the user is already present with the user ID X then add Y dollars to their account.
 - You need to return true only when you create a new account (which means a user with a user ID X is not present); otherwise, return false.
- **DEBIT $X\ Y$**
 - Debit Y dollars from the user's bank account with user ID X .
 - If the user with user ID X is not present or has less than Y dollars, then do nothing and return false.
 - Otherwise, debit Y dollars and return true.
- **CREDIT $X\ Y$**
 - Credit Y dollars to the user's bank account with user ID X .
 - If the user with user ID X is not present, then do nothing and return false.
 - Otherwise, credit Y dollars and return true.
- **BALANCE X**
 - Return the user's bank balance with the user ID X .

- If the user with user ID X is not present, then return -1.

Class description

- Bank: Represents information about the users
- Public functions: Refers to the types of queries (described above) that must be implemented

Input format for custom testing

Note: Use this input format if you are testing against custom input or writing code in a language where we don't provide boilerplate code.

- The first line contains a single integer Q , denoting the number of elements in the array.
- Each of the next Q lines contains the query (case-sensitive for strings).

Output format

Return the result for each function method based on the query.

Assignment 4

Vibe Coding – Deadline 03-05-2026

Dear Students,

- Complete the assigned task using AI tools (ChatGPT, Copilot, cursor etc.)
- Focus on how you interact with AI (your prompts), not just the final code
- Submit the following:
 1. Your final code file (.cpp)
 2. A document/screenshots of prompts you used (clearly showing your interaction with AI)
- Prompts should reflect your understanding and problem-solving approach
- Do not copy prompts from others; originality is important
- Follow file naming format:
 1. QuestionName.cpp
 2. QuestionName_Prompts.pdf / .png
- Late or incomplete submissions will not be evaluated
- Evaluation will be based on:
 1. Quality of prompts (clarity, logic, iteration)
 2. Understanding of code generated
 3. Final output correctness

Submissions without prompts will not be considered

7. AI Roast Generator

Students will build a console-based program that generates funny roasts based on a user's name. The program should produce different roasts each time it runs by randomly selecting from multiple roast templates. For example: If the Input is Rahul, then the possible output can be:

- "Rahul writes code so slow that even a turtle switched to Python."
- "If procrastination were an Olympic sport, Rahul would already have a gold medal."
- "Rahul's debugging style is basically staring at the screen until the bug gets embarrassed and leaves."

What Students Have to Do

Students should create a program that:

- Takes user input (name)
- Stores multiple roast templates
- Uses random selection to generate different roasts
- Replaces placeholders like {name} with the actual input
- Displays a unique roast message

The program should have at least 8–10 roast templates.

Example template format:

"{name}'s code runs so slowly that even dial-up internet feels fast."

"If laziness had a brand ambassador, it would be {name}."

"{name} doesn't debug code — they negotiate with bugs."

Integration of Vibe Coding

Students must use AI coding tools such as ChatGPT, GitHub Copilot, Codeium, Claude, or Gemini during development. The goal is to experience Vibe Coding, where developers describe the idea in natural language and the AI helps generate, improve, and debug the program.

Students should use AI tools for:

- Generating program logic
- Creating roast templates
- Improving humour or creativity
- Fixing bugs in the code
- Refactoring or improving the structure

Students should interact with the AI through clear prompts, refine those prompts, and observe how the generated program improves.

Bug Identification and Fixing (Important)

Students must demonstrate at least one bug-fixing interaction using AI.

Possible bugs may include:

- Roasts not changing (random function issue)
- Placeholder {name} not being replaced

- Program crashing on empty input
- Incorrect input reading

Students should:

1. Run the generated program
2. Identify a bug or unexpected behavior
3. Ask the AI to fix it
4. Apply the corrected code

Students must capture screenshots of this debugging process.

Deliverables

Students must submit the following:

1. AI Prompt Used

Students must submit one well-written prompt that they used to generate the program.

Example of a good prompt:

“Write a beginner-friendly C++ console program that generates funny roast messages.

The program should ask the user for their name.

Store at least 10 roast templates in a vector.

Randomly select a roast and replace placeholder {name} with the user’s input.

Ensure the program produces different roasts each time it runs.

Add comments so beginners can understand the code.”

Students are encouraged to refine the prompt multiple times to improve the output.

2. Screenshots of Bug Fixing using AI

Students should submit:

- Screenshot of the bug or incorrect output
- Screenshot of the AI prompt used to fix the bug
- Screenshot of the corrected output

8. Student Excuse Generator

Students will build a console-based program that generates creative excuses for missing homework or assignments. The program should generate different excuses each time it runs. For example: If the Input is Priya, then the possible output.

- “Priya couldn’t finish the assignment because the laptop decided to install updates for six hours.”
- “Priya was about to complete the homework when the Wi-Fi mysteriously disappeared.”
- “Priya tried finishing the assignment, but the computer keyboard suddenly stopped cooperating.”

Create a program that:

- Takes user input (student name)
- Stores multiple excuse templates
- Uses random selection to generate excuses
- Replaces placeholders like {name} with the actual input
- Displays a random excuse

The program should have at least 10 excuse templates.

Example template format:

"{name} couldn't complete the assignment because the laptop battery died unexpectedly."

"{name} tried finishing the homework, but the internet stopped working."

"{name} was ready to submit the assignment when the file mysteriously disappeared."

Integration of Vibe Coding

Students must use AI coding tools such as ChatGPT, GitHub Copilot, Codeium, Claude, or Gemini.

Students should use AI tools for:

- Generating excuse templates
- Improving creativity of excuses
- Fixing bugs in the code
- Improving the program logic

Students should refine prompts and observe how AI responses improve.

Bug Identification and Fixing (Important)

Students must demonstrate at least one bug-fixing interaction using AI.

Possible bugs may include:

- Excuses repeating every time
- Placeholder {name} not replaced correctly
- Program not reading input properly

Students should:

1. Run the program
2. Identify a bug
3. Ask AI to fix the issue
4. Apply the correction

Students must capture screenshots of the debugging process.

Deliverables

1. AI Prompt Used

Example prompt:

"Write a beginner-friendly C++ console program that generates random student excuses.

The program should ask for the student's name.

Store at least 10 excuse templates in a vector and randomly select one.

Replace {name} with the user input.

Ensure different excuses appear each time the program runs."

Students are encouraged to refine the prompt.

2. Screenshots of Bug Fixing using AI

Students should submit:

- Screenshot of incorrect output
- Screenshot of AI prompt used to fix the bug
- Screenshot of corrected output